From Recursion to Loop: `BinarySearchLoop`

Practice: nonrecursive binary search (Exercise 4.2.2), step by step

Marcus Birkenkrahe

COSC 2230 · CS III · Augustana University

Fall 2026 · Sedgewick & Wayne, *Introduction to Programming in Java* 2e, §4.2

Why this exercise

- **Your first recursion-to-loop rewrite.** Binary search is just the example: you will need this again for mergesort, stacks (4.3), and recursions that get too deep

Why this exercise

- **Your first recursion-to-loop rewrite.** Binary search is just the example: you will need this again for mergesort, stacks (4.3), and recursions that get too deep
- **The half-open interval `a[lo, hi)` made concrete.** The loop condition, the new `lo` or `hi`, and the `-1` after the loop are all questions about the interval

Why this exercise

- **Your first recursion-to-loop rewrite.** Binary search is just the example: you will need this again for mergesort, stacks (4.3), and recursions that get too deep
- **The half-open interval `a[lo, hi)` made concrete.** The loop condition, the new `lo` or `hi`, and the `-1` after the loop are all questions about the interval
- **What real code does.** Java's own `Arrays.binarySearch` is a loop. Loop invariant: if `key` is in `a`, it is in `a[lo, hi)`

Why this exercise

- **Your first recursion-to-loop rewrite.** Binary search is just the example: you will need this again for mergesort, stacks (4.3), and recursions that get too deep
- **The half-open interval `a[lo, hi)` made concrete.** The loop condition, the new `lo` or `hi`, and the `-1` after the loop are all questions about the interval
- **What real code does.** Java's own `Arrays.binarySearch` is a loop. Loop invariant: if `key` is in `a`, it is in `a[lo, hi)`
- **Binary search itself doesn't need it.** The recursion is only about lg N deep – about 30 for a billion entries

Why this exercise

- **Your first recursion-to-loop rewrite.** Binary search is just the example: you will need this again for mergesort, stacks (4.3), and recursions that get too deep
- **The half-open interval `a[lo, hi)` made concrete.** The loop condition, the new `lo` or `hi`, and the `-1` after the loop are all questions about the interval
- **What real code does.** Java's own `Arrays.binarySearch` is a loop. Loop invariant: if `key` is in `a`, it is in `a[lo, hi)`
- **Binary search itself doesn't need it.** The recursion is only about lg N deep – about 30 for a billion entries
- **Do it by hand, without AI.** Any chatbot writes this in seconds; the point is that you can

Practice: BinarySearchLoop (15 min)

→ **Switch to editor · BinarySearchLoop.java, test client from class**

```
public static int search(String key, String[] a)
{
    return search(key, a, 0, a.length);
}
public static int search(String key, String[] a,
                         int lo, int hi)
{
    if (hi <= lo) return -1;
    int mid = lo + (hi - lo) / 2;
    int cmp = a[mid].compareTo(key);
    if      (cmp > 0) return search(key, a, lo, mid);
    else if (cmp < 0) return search(key, a, mid+1, hi);
    else              return mid;
}
```

Task: write one method `search(String key, String[] a)` that does the same with a `while` loop instead of recursion.

Practice: BinarySearchLoop (15 min)

→ **Switch to editor · BinarySearchLoop.java, test client from class**

```
public static int search(String key, String[] a)
{
    return search(key, a, 0, a.length);
}
public static int search(String key, String[] a,
                         int lo, int hi)
{
    if (hi <= lo) return -1;
    int mid = lo + (hi - lo) / 2;
    int cmp = a[mid].compareTo(key);
    if      (cmp > 0) return search(key, a, lo, mid);
    else if (cmp < 0) return search(key, a, mid+1, hi);
    else              return mid;
}
```

Task: write one method `search(String key, String[] a)` that does the same with a `while` loop instead of recursion.

We build it in five steps. Write one line per step, then compare with your neighbor.

Practice: BinarySearchLoop (15 min)

→ **Switch to editor · BinarySearchLoop.java, test client from class**

```
public static int search(String key, String[] a)
{
    return search(key, a, 0, a.length);
}
public static int search(String key, String[] a,
                         int lo, int hi)
{
    if (hi <= lo) return -1;
    int mid = lo + (hi - lo) / 2;
    int cmp = a[mid].compareTo(key);
    if      (cmp > 0) return search(key, a, lo, mid);
    else if (cmp < 0) return search(key, a, mid+1, hi);
    else              return mid;
}
```

Task: write one method `search(String key, String[] a)` that does the same with a `while` loop instead of recursion.

We build it in five steps. Write one line per step, then compare with your neighbor.

Closed book, no AI.

Step 1 · Which parameters change?

```
if (cmp > 0)      return search(key, a, lo, mid);
else if (cmp < 0) return search(key, a, mid+1, hi);
```

Compare the arguments of the two recursive calls with the parameters of the current call `search(key, a, lo, hi)`. Which change from call to call, and which stay the same?

Step 1 · Which parameters change?

```
if (cmp > 0)      return search(key, a, lo, mid);
else if (cmp < 0) return search(key, a, mid+1, hi);
```

Compare the arguments of the two recursive calls with the parameters of the current call `search(key, a, lo, hi)`. Which change from call to call, and which stay the same?

→ `key` and `a` stay the same. `lo` and `hi` change: only the interval `a[lo, hi)` shrinks.

Step 1 · Which parameters change?

```
if (cmp > 0)      return search(key, a, lo, mid);
else if (cmp < 0) return search(key, a, mid+1, hi);
```

Compare the arguments of the two recursive calls with the parameters of the current call `search(key, a, lo, hi)`. Which change from call to call, and which stay the same?

→ `key` and `a` stay the same. `lo` and `hi` change: only the interval `a[lo, hi)` shrinks.

→ So `lo` and `hi` become **local variables** that the loop updates. `key` and `a` stay parameters.

Step 2 · Where do `lo` and `hi` start?

```
public static int search(String key, String[] a)
{
    return search(key, a, 0, a.length);
}
```

In the loop version there are no recursive calls to pass `lo` and `hi` along. Where do you read off their first values?

Step 2 · Where do `lo` and `hi` start?

```
public static int search(String key, String[] a)
{
    return search(key, a, 0, a.length);
}
```

In the loop version there are no recursive calls to pass `lo` and `hi` along. Where do you read off their first values?

→ From the call that starts the recursion: `search(key, a, 0, a.length)`. The whole array is the first interval.

Step 2 · Where do `lo` and `hi` start?

```
public static int search(String key, String[] a)
{
    return search(key, a, 0, a.length);
}
```

In the loop version there are no recursive calls to pass `lo` and `hi` along. Where do you read off their first values?

→ From the call that starts the recursion: `search(key, a, 0, a.length)`. The whole array is the first interval.

```
public static int search(String key, String[] a)
{
    int lo = 0, hi = a.length;
}
```

Step 3 · When does the loop stop?

```
if (hi <= lo) return -1;
```

The recursion stops here. Turn it around: as long as what is true should the loop keep going? And where does “key not found” end up?

Step 3 · When does the loop stop?

```
if (hi <= lo) return -1;
```

The recursion stops here. Turn it around: as long as what is true should the loop keep going? And where does "key not found" end up?

→ `while (hi > lo)` – the interval is not empty. Strict >: `lo == hi` is the empty interval.

Step 3 · When does the loop stop?

```
if (hi <= lo) return -1;
```

The recursion stops here. Turn it around: as long as what is true should the loop keep going? And where does "key not found" end up?

→ `while (hi > lo)` – the interval is not empty. Strict >: `lo == hi` is the empty interval.

→ Two ways out: `return mid` (found), or the condition turns false (not found). Then the line after the loop runs: `return -1;`

Step 3 · When does the loop stop?

```
if (hi <= lo) return -1;
```

The recursion stops here. Turn it around: as long as what is true should the loop keep going? And where does "key not found" end up?

→ `while (hi > lo)` – the interval is not empty. Strict >: `lo == hi` is the empty interval.

→ Two ways out: `return mid` (found), or the condition turns false (not found). Then the line after the loop runs: `return -1;`

```
public static int search(String key, String[] a)
{
    int lo = 0, hi = a.length;
    while (hi > lo)
    {
    }
    return -1;
}
```

Step 4 · What does each call do to `lo` or `hi`?

	key	a	lo	hi
current call	key	a	lo	hi
`search(key, a, lo, mid)`	key	a	lo	mid
`search(key, a, mid+1, hi)`	key	a	mid+1	hi

Step 4 · What does each call do to `lo` or `hi`?

	key	a	lo	hi
current call	key	a	lo	hi
`search(key, a, lo, mid)`	key	a	lo	**mid**
`search(key, a, mid+1, hi)`	key	a	**mid+1**	hi

→ Only one position changes per call. Each call becomes **one assignment**: `hi = mid;` and `lo = mid + 1;`

Step 4 · What does each call do to `lo` or `hi`?

	key	a	lo	hi
current call	key	a	lo	hi
`search(key, a, lo, mid)`	key	a	lo	**mid**
`search(key, a, mid+1, hi)`	key	a	**mid+1**	hi

→ Only one position changes per call. Each call becomes **one assignment**: `hi = mid;` and `lo = mid + 1;`

→ No temporary variable needed (unlike gcd): each branch changes only one variable.

Step 4 · What does each call do to lo or hi?

	key	a	lo	hi
current call	key	a	lo	hi
search(key, a, lo, mid)	key	a	lo	**mid**
search(key, a, mid+1, hi)	key	a	**mid+1**	hi

→ Only one position changes per call. Each call becomes **one assignment**: `hi = mid;` and `lo = mid + 1;`

→ No temporary variable needed (unlike gcd): each branch changes only one variable.

```
int lo = 0, hi = a.length;
while (hi > lo)
{
    int mid = lo + (hi - lo) / 2;
    int cmp = a[mid].compareTo(key);
    if      (cmp > 0) hi = mid;
    else if (cmp < 0) lo = mid + 1;
}
return -1;
```

Step 5 · What happens to `return mid;`?

The third branch, `else return mid;`, is not a recursive call. Where does it go?

Step 5 · What happens to `return mid;`?

The third branch, `else return mid;`, is not a recursive call. Where does it go?

→ It stays as it is, inside the loop. When the key is found, `return` ends the loop and the method at once.

```
int lo = 0, hi = a.length;
while (hi > lo)
{
    int mid = lo + (hi - lo) / 2;
    int cmp = a[mid].compareTo(key);
    if      (cmp > 0) hi = mid;
    else if (cmp < 0) lo = mid + 1;
    else              return mid;
}
return -1;
```

Step 5 · What happens to `return mid;`?

The third branch, `else return mid;`, is not a recursive call. Where does it go?

→ It stays as it is, inside the loop. When the key is found, `return` ends the loop and the method at once.

Now run the test client from class.

```
int lo = 0, hi = a.length;
while (hi > lo)
{
    int mid = lo + (hi - lo) / 2;
    int cmp = a[mid].compareTo(key);
    if      (cmp > 0) hi = mid;
    else if (cmp < 0) lo = mid + 1;
    else              return mid;
}
return -1;
```

Same tests, same results

```
public static int search(String key, String[] a)
{
    int lo = 0, hi = a.length;
    while (hi > lo)
    {
        int mid = lo + (hi - lo) / 2;
        int cmp = a[mid].compareTo(key);
        if      (cmp > 0) hi = mid;
        else if (cmp < 0) lo = mid + 1;
        else              return mid;
    }
    return -1;
}
```

Same tests, same results

```
public static int search(String key, String[] a)
{
    int lo = 0, hi = a.length;
    while (hi > lo)
    {
        int mid = lo + (hi - lo) / 2;
        int cmp = a[mid].compareTo(key);
        if      (cmp > 0) hi = mid;
        else if (cmp < 0) lo = mid + 1;
        else              return mid;
    }
    return -1;
}
```

Test client from class:

```
all 15 hits found
aaron  -> -1
marvin -> -1
zoe    -> -1
empty, eve -> -1
one,   eve -> 0
one,   bob -> -1
```

Same tests, same results

```
public static int search(String key, String[] a)
{
    int lo = 0, hi = a.length;
    while (hi > lo)
    {
        int mid = lo + (hi - lo) / 2;
        int cmp = a[mid].compareTo(key);
        if      (cmp > 0) hi = mid;
        else if (cmp < 0) lo = mid + 1;
        else              return mid;
    }
    return -1;
}
```

Test client from class:

```
all 15 hits found
aaron  -> -1
marvin -> -1
zoe    -> -1
empty, eve -> -1
one,   eve -> 0
one,   bob -> -1
```

→ Identical to the recursive version.

Why does the book use recursion?

Same authors, two books, two versions – why?

Why does the book use recursion?

Same authors, two books, two versions – why?

- **The book teaches.** The recursive code says what the algorithm does: look at the middle, then search the left or the right half

Why does the book use recursion?

Same authors, two books, two versions – why?

- **The book teaches.** The recursive code says what the algorithm does: look at the middle, then search the left or the right half
- **Analysis and proof follow the code.** $T(N) = T(N/2) + 1$ (p. 535): one compare plus one call on half the interval. Correctness: induction on the interval size

Why does the book use recursion?

Same authors, two books, two versions – why?

- **The book teaches.** The recursive code says what the algorithm does: look at the middle, then search the left or the right half
- **Analysis and proof follow the code.** $T(N) = T(N/2) + 1$ (p. 535): one compare plus one call on half the interval. Correctness: induction on the interval size
- **It fits the chapter.** 4.2 builds on recursion (2.3), and mergesort comes next. Exercise 4.2.2 asks for the loop, so the book wants you to see both

Why does the book use recursion?

Same authors, two books, two versions – why?

- **The book teaches.** The recursive code says what the algorithm does: look at the middle, then search the left or the right half
- **Analysis and proof follow the code.** $T(N) = T(N/2) + 1$ (p. 535): one compare plus one call on half the interval. Correctness: induction on the interval size
- **It fits the chapter.** 4.2 builds on recursion (2.3), and mergesort comes next. Exercise 4.2.2 asks for the loop, so the book wants you to see both
- **The library has to be fast.** Every call costs a stack frame, and Java does not turn a call that comes last into a loop by itself. Not because of stack overflow: the depth is only lg N

Why does the book use recursion?

Same authors, two books, two versions – why?

- **The book teaches.** The recursive code says what the algorithm does: look at the middle, then search the left or the right half
- **Analysis and proof follow the code.** $T(N) = T(N/2) + 1$ (p. 535): one compare plus one call on half the interval. Correctness: induction on the interval size
- **It fits the chapter.** 4.2 builds on recursion (2.3), and mergesort comes next. Exercise 4.2.2 asks for the loop, so the book wants you to see both
- **The library has to be fast.** Every call costs a stack frame, and Java does not turn a call that comes last into a loop by itself. Not because of stack overflow: the depth is only lg N
- Sedgewick & Wayne, *Algorithms* 4e – a book about efficient implementations – writes `BinarySearch` with a `while` loop